```
# Create a .txt file containing all 100 advanced Python OOP formulas
formulas = """
51. Enforcing Interface with Abstract Base Class
from abc import ABC, abstractmethod
class Interface(ABC):
    @abstractmethod
    def method(self):
        pass

52. Prevent Instantiation of a Class
class Utility:
    def __new__(cls):
        raise NotImplementedError("Can't instantiate")

53. Property with Getter and Setter
class Temperature:
    def __init__(self):
        self._celsius = 0

    @property
    def celsius(self):
        return self._celsius

    @celsius.setter
    def celsius(self, value):
        self._celsius = value

54. Dynamically Add Property
```

setattr(obj, 'new_attr', 10)

55. Dynamically Remove Property

delattr(obj, 'new_attr')

56. Use `__dict__` for Object Introspection

print(obj.__dict__)

57. Shallow Copy of Object

import copy

copy.copy(obj)

58. Deep Copy of Object

copy.deepcopy(obj)

59. Freeze Class (Immutable After Init)

```
class Frozen:
  __isfrozen = False
  def __setattr__(self, key, value):
    if self.__isfrozen:
      raise TypeError("Frozen")
    super().__setattr__(key, value)
  def freeze(self):
    self.__isfrozen = True
```

60. Register Class as Plugin

```
registry = {}
def register(cls):
```

```
    registry[cls.__name__] = cls
    return cls
@register
class Plugin:
    pass
```

61. Custom `__getattr__` for Fallback

```
class Fallback:
    def __getattr__(self, name):
        return f"{name} not found"
```

62. `__del__` for Destructor

```
class Sample:
    def __del__(self):
        print("Object deleted")
```

63. Dynamic Class from String

```
cls = globals()["MyClass"]
obj = cls()
```

64. Apply Multiple Decorators

```
@dec1
@dec2
def func(): pass
```

65. Check if Attribute Exists

```
hasattr(obj, 'attr')
```

66. Prevent Attribute Deletion

```
def __delattr__(self, item):
    raise AttributeError("Can't delete")
```

67. Add Class Constant

```
class Config:
    PI = 3.14
```

68. Method Chaining with Return self

```
def set_value(self, val):
    self.val = val
    return self
```

69. Custom `__contains__` for `in`

```
def __contains__(self, item):
    return item in self.data
```

70. Custom `__eq__` for ==

```
def __eq__(self, other):
    return self.id == other.id
```

71. Custom `__lt__`, `__gt__`, etc.

```
def __lt__(self, other):
    return self.score < other.score
```

72. Enforce Class Method Call Only

```
class OnlyClass:
    @classmethod
```

```
  def method(cls):
    print("Class method")
```

73. Add Logging to All Methods (Meta)

```
class LogMeta(type):
  def __new__(cls, name, bases, dct):
    for key, val in dct.items():
      if callable(val):
        def wrap(fn):
          def inner(*a, **k):
            print(f"Calling {fn.__name__}")
            return fn(*a, **k)
          return inner
        dct[key] = wrap(val)
    return type.__new__(cls, name, bases, dct)
```

74. Validate Attribute Types

```
class Person:
  def __init__(self, age: int):
    if not isinstance(age, int):
      raise TypeError
```

75. Apply Decorator to All Methods

```
def method_logger(method):
  def wrapper(*args, **kwargs):
    print(f"Calling {method.__name__}")
    return method(*args, **kwargs)
  return wrapper
```

76. Enforce Subclass Override

```
class Base:
    def method(self):
        raise NotImplementedError
```

77. Dynamically Add Method

```
def new_method(self):
    print("Added!")
obj.method = MethodType(new_method, obj)
```

78. Object Equality via `__hash__`

```
def __hash__(self):
    return hash((self.id, self.name))
```

79. Use Enum in Class

```
from enum import Enum
class Color(Enum):
    RED = 1
```

80. Auto Increment ID

```
class AutoID:
    counter = 0
    def __init__(self):
        AutoID.counter += 1
        self.id = AutoID.counter
```

81. Restrict Subclassing

```
class Final:
    def __init_subclass__(cls, **kwargs):
        raise TypeError("No subclassing allowed")
```

82. Callable Object with State

```
class Counter:
    def __init__(self):
        self.count = 0
    def __call__(self):
        self.count += 1
        return self.count
```

83. Proxy Pattern

```
class RealObject:
    def operation(self): return "real"

class Proxy:
    def __init__(self):
        self.obj = RealObject()
    def operation(self): return self.obj.operation()
```

84. Null Object Pattern

```
class Null:
    def __getattr__(self, name):
        return self
    def __call__(self, *args, **kwargs):
        return self
```

85. Flyweight Pattern

```
class Flyweight:
    _instances = {}
    def __new__(cls, key):
        if key not in cls._instances:
            cls._instances[key] = super().__new__(cls)
        return cls._instances[key]
```

86. Mediator Pattern

```
class Mediator:
    def __init__(self):
        self.components = []

    def notify(self, sender, event):
        for comp in self.components:
            if comp != sender:
                comp.receive(event)
```

87. Chain of Responsibility

```
class Handler:
    def __init__(self, next_handler=None):
        self.next = next_handler
    def handle(self, request):
        if self.next:
            return self.next.handle(request)
```

88. Template Method Pattern

```
class Template:
```

```
  def run(self):
    self.step1()
    self.step2()
  def step1(self): pass
  def step2(self): pass
```

89. Custom Iterable

```
class MyRange:
  def __init__(self, end):
    self.i = 0
    self.end = end
  def __iter__(self): return self
  def __next__(self):
    if self.i < self.end:
      val = self.i
      self.i += 1
      return val
    raise StopIteration
```

90. Use `__format__`

```
def __format__(self, format_spec):
  return f"{self.name:{format_spec}}"
```

91. Lazy Initialization

```
class Lazy:
  def __init__(self):
    self._data = None
  @property
```

```
    def data(self):
        if self._data is None:
            self._data = "Loaded"
        return self._data
```

92. Dynamic Import in Class

```
mod = __import__('math')
print(mod.sqrt(16))
```

93. Method Dispatching Based on Type

```
from functools import singledispatchmethod
class Demo:
    @singledispatchmethod
    def process(self, arg):
        print("Default")

    @process.register
    def _(self, arg: int):
        print("Integer")
```

94. Inner/Nested Class

```
class Outer:
    class Inner:
        def greet(self):
            return "Hi"
```

95. Yield Inside Method

```
class Counter:
```

```
  def generator(self):
    for i in range(5):
      yield i
```

96. Class With Iterator

```
class Iterable:
  def __iter__(self):
    return iter([1, 2, 3])
```

97. Clone via `__copy__`

```
def __copy__(self):
  return type(self)(self.name)
```

98. Clone via `__deepcopy__`

```
def __deepcopy__(self, memo):
  return type(self)(copy.deepcopy(self.name, memo))
```

99. Init Class from Dict

```
class Person:
  def __init__(self, **kwargs):
    self.__dict__.update(kwargs)
```

100. Override `__bool__`

```
def __bool__(self):
  return bool(self.name)
"""

file_path = "/mnt/data/advanced_python_oop_51_to_100.txt"
```

```
with open(file_path, "w") as f:
    f.write(formulas)


file_path
```